DSA03-NATURAL LANGUAGE PROCESSING ASSIGNMENT-1

ASSIGNMENT 1 – CO1

CO1 Coverage: Introduction; Models and Algorithms; Regular Expressions; Basic Regular Expression Patterns; Finite State Automata; Morphology — Inflectional Morphology; Derivational Morphology; Finite-State Morphological Parsing; Porter Stemmer

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: 4-11-2026
Name: Syed Nazma(192425193)

Assignment Title: Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System Using Regular Expressions, Finite-State Automata and Morphological Parsing (Total Marks – 50)

Duration: 1 week

Scenario — Text Normalisation and Search for a Hospital Record System

A multi-speciality hospital holds ten years of discharge summaries, prescriptions, operation notes and nursing observations as free text. Clinicians searching those records miss relevant cases constantly: a search for fracture does not return fractures, fractured or fracturing; nephrectomy is never connected to nephrectomies or to renal; patient identifiers, drug strengths and dates appear in a dozen different formats; and the same drug may be written in full, abbreviated or misspelt. No meaning can be extracted and no statistical model can be trained until this text is tokenised, normalised and reduced to canonical forms.

You are the language-engineering team. Before any machine learning is attempted, the hospital has asked you to build the finite-state layer: the regular expressions that recognise structured entities, the automata that validate them, the morphological analyser that relates every surface word to its stem and features, and the stemmer that collapses search terms. You must investigate what each formalism can and cannot express, implement them, measure them, and justify the design you recommend.

Planning data and constraints

- Corpus: at least 500 de-identified clinical records (real or synthetic, but documented), totalling at least 25,000 word tokens and covering at least four record types.
- Pattern inventory: at least 12 structured entity types to be recognised by regular expression — patient identifier, admission and discharge date, drug name with

strength, dosage frequency, laboratory value with unit, blood pressure, temperature, ICD code, ward and bed, contact number, age, and duration of symptom.

- Automata: at least four finite-state automata, each with at least six states, including one entity format that cannot be recognised conveniently without non-determinism.
- Morphological resources (build and document your own): a lexicon of at least 300 stems, at least 20 inflectional affixes and at least 15 derivational affixes, with irregular forms listed separately.
- Evaluation data: at least 40 clinician queries with relevance judgements over the corpus, and at least 200 word forms hand-annotated with their stem and features.
- Any manual effort (pattern writing, lexicon building, annotation, testing) must be accounted for in your effort and cost discussion.

Design Challenge (Day 4):

The system must now ingest notes captured by optical character recognition from scanned handwritten forms, and notes typed in Romanised regional language mixed with English. Re-evaluate your original patterns, automata and analyser and decide whether to retain, modify or redesign them.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis 3 Marks

Guiding Enquiry

Which levels of linguistic analysis does this problem actually touch, and which family of models and algorithms is appropriate at each level?

Investigate and interpret:

- Describe the levels of linguistic analysis — phonology, morphology, syntax, semantics, pragmatics and discourse — and state, with an example drawn from your own corpus, which levels the record-search problem requires and which it can safely ignore.
- Distinguish the three model families used throughout this unit: state machines, formal rule systems and probabilistic models. Name one task in the hospital problem that each family suits best and say why the other two would be the wrong choice for it.
- Explain why ambiguity is the central problem of natural language processing, and identify at least four distinct ambiguities present in your corpus — lexical, morphological, segmentation and abbreviation ambiguity.
- Identify where search appears in this unit (state-space search in automata, backtracking in matching) and state the resources each consumes: time, memory, annotated data and expert effort.
- Set out, as a labelled diagram, the pipeline you will build in the remaining tasks, naming the input, the output and the formalism of every stage.

Evidence Expected

A reasoned problem framing that maps the hospital task onto levels of analysis, model families and a concrete, staged pipeline.

TASK 2 – Basic Regular Expression Patterns 4 Marks

Guiding Enquiry

What can the basic regular expression operators express, and how are they combined into patterns that match real clinical strings exactly?

Investigate and interpret:

- Define the basic operators — concatenation, disjunction, Kleene star and plus, optionality, character classes, ranges, negated classes, anchors and counters — and give one minimal clinical example of each.
- Write and document a regular expression for each of the twelve entity types in your pattern inventory, stating for every pattern both what it must match and what it must not.
- Demonstrate greedy and non-greedy matching on a clinical sentence containing two dosage expressions, and explain precisely why the two quantifiers return different spans.
- Explain operator precedence and grouping, and show with a worked example how a misplaced disjunction silently changes the language the expression denotes.
- Evaluate every pattern over the corpus, reporting true positives, false positives and false negatives, and name the specific linguistic cause of each error class.

Evidence Expected

A documented pattern inventory with precise match specifications and a per-pattern error analysis carried out on real data.

TASK 3 – Regular Expressions for Tokenisation and Normalisation 4 Marks

Guiding Enquiry

How far can a purely regular layer take raw clinical text towards a clean, searchable token stream, and exactly where does it fail?

Investigate and interpret:

- Specify your tokenizer as a single regular expression, defining precisely what counts as a token — hyphenated terms, abbreviations containing full stops, decimal values, units written against numbers — and justify each decision.
- Apply maximal-munch (longest-match) tokenisation and show on at least three examples how the result differs from a naive split on whitespace.
- Build substitution rules that fold case, expand at least thirty clinical abbreviations and canonicalise date and unit formats; show the complete before-and-after text of one full record.
- Identify at least three normalisation decisions that are irreversible because information is destroyed, and argue when that loss is acceptable and when it is not.
- Give one requirement of the hospital task that no regular expression can express, and argue informally why — for example, matching balanced parentheses in a nested clinical note.

Evidence Expected

A working regular-expression tokenizer and normaliser with worked examples and a clear statement of the limits of regularity.

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence 5 Marks

Guiding Enquiry

How do automata recognise exactly the same languages as regular expressions, and what does removing non-determinism cost?

Investigate and interpret:

- Define a finite automaton formally as the five-tuple (Q, Σ, δ, q_0, F), then design a deterministic automaton for at least four validators from your pattern inventory, giving both the transition diagram and the transition table for each.
- Design a non-deterministic automaton with ε-transitions for one entity whose format has optional parts, and trace its behaviour on one accepted and one rejected string, showing the set of states live at every step.
- Convert that NFA to a DFA by subset construction, showing the complete construction table, then minimise the DFA and report the state counts before and after.
- Apply Thompson's construction to turn one Task 2 regular expression into an automaton, and derive a regular expression back from one of your automata, demonstrating the equivalence in both directions.
- State the pumping lemma for regular languages and use it to prove that one specific clinical pattern is not regular — for example a note requiring equal numbers of opening and closing brackets — and relate the proof to your finding in Task 3.

Evidence Expected

Correct automata with full subset-construction and minimisation tables, a demonstrated two-way equivalence with regular expressions, and a valid non-regularity proof.

TASK 5 – Inflectional Morphology 3 Marks

Guiding Enquiry

Which inflectional processes appear in clinical English, and how are surface forms systematically related to their stems?

Investigate and interpret:

- Define morpheme, stem, affix, inflection and paradigm, and separate inflection from derivation using the standard criteria of category preservation, productivity and position relative to other affixes.
- List every inflectional paradigm present in your corpus — noun number, verb tense and aspect, subject agreement, adjective comparison — and give the full paradigm for at least five clinical lexemes.
- State the orthographic rules governing English inflection — e-deletion, consonant doubling, y-to-i, the sibilant -es rule — precisely enough to be implemented, and give for each one a counter-example on which it over-generates.
- Identify the irregular forms in your corpus and explain why each must be listed in the lexicon rather than derived by rule.
- Measure the effect of inflection: report the type-to-lemma ratio of your corpus and the proportion of failures in your query set that inflection alone explains.

Evidence Expected

Precisely stated inflectional rules with paradigms, an irregular list, and a quantified measurement of the effect of inflection on retrieval.

TASK 6 – Derivational Morphology 4 Marks

Guiding Enquiry

How does derivation build the specialised clinical vocabulary, and why is it harder to handle than inflection?

Investigate and interpret:

- Define derivation and list at least fifteen derivational affixes attested in your corpus — for example -itis, -ectomy, -ology, -osis, -aemia, -ation, -able, -ive, nephro-, hyper-, hypo-, re-, un- — stating the category change each one causes.
- Build the derivational family of at least six clinical roots, showing the order in which affixes attach and identifying at least two orders that are impossible.
- Explain semantic non-compositionality with clinical examples in which the meaning of the derived word cannot be predicted from its parts, and state what this implies for a search system that stems aggressively.
- Compare the productivity of your affixes empirically: count types and tokens for each in the corpus and classify each affix as productive or fossilised, with the evidence for the classification.
- Decide whether derivational relatives should be conflated for search, citing at least two cases where conflation helps the clinician and two where it produces a clinically misleading match.

Evidence Expected

A documented derivational analysis with an affix inventory, productivity measurements and a reasoned position on conflation.

TASK 7 – Finite-State Morphological Parsing 5 Marks

Guiding Enquiry

How can a transducer relate a surface form to its stem and features, and run in both directions on the same machine?

Investigate and interpret:

- Define a finite-state transducer as a two-tape automaton and explain the lexical and surface tapes with one fully worked clinical example.
- Build your lexicon as an FST organised into continuation classes, covering at least three parts of speech together with the inflectional and derivational affixes of Tasks 5 and 6.
- Hand-trace the parsing of at least five surface forms, showing the state sequence, the symbols consumed and emitted on each tape, and the final analysis — for example fractures → fracture +N +PL.
- Run the same transducer in the generation direction, producing surface forms from stem-plus-feature input, and show explicitly that parsing and generation use one machine and not two.
- Handle ambiguity: give at least two surface forms with more than one analysis, show the transducer returning all of them, and state how a downstream component would choose between them.

Evidence Expected

A working transducer specification with traced analyses in both directions and an explicit treatment of ambiguity.

TASK 8 – Orthographic Rules and Two-Level Morphology 4 Marks

Guiding Enquiry

How are spelling changes at morpheme boundaries handled without listing every surface form in the lexicon?

Investigate and interpret:

- Explain why a lexicon transducer alone cannot produce correct surface spellings, using at least three examples in which plain concatenation yields the wrong string.
- Write at least five orthographic rules in the standard rewrite notation (a → b / c ___d), stating the context of each rule precisely.
- Implement each rule as a transducer and compose the rule transducers with the lexicon FST; draw the cascade and state the order dependencies between the rules.
- Contrast this sequential cascade with the parallel two-level (Koskenniemi) formulation, and explain the practical advantage each has over the other.
- Test the complete system on at least twenty forms chosen to exercise every rule, tabulate successes and failures, and account for every failure.

Evidence Expected

Correct rule transducers, a documented composition order, and a tested cascade with full error accounting.

TASK 9 – The Porter Stemmer 5 Marks

Guiding Enquiry

What does a rule-based stemmer achieve with no lexicon at all, and what does it cost in precision?

Investigate and interpret:

- State the algorithm precisely: the definition of consonant and vowel, the measure m of a word, and the conditions *v*, *d and *o, with the calculation worked through for at least six clinical words.
- Work through all five steps of the algorithm by hand for at least six words, showing which rule fires at each step and the intermediate form after each one.
- Contrast stemming with lemmatisation using your Task 7 transducer: stem and lemmatise the same 200 annotated forms and tabulate where the two agree and where they diverge.
- Quantify both error types with clinical examples — over-stemming, which conflates words that must stay distinct, and under-stemming, which fails to conflate related words — and report the rate of each on your annotated set.
- Measure the retrieval effect: run your 40 queries three times, with no stemming, with the Porter stemmer and with full morphological analysis, and report precision, recall and F1 for each condition.

Evidence Expected

Correct hand-traces of the algorithm, a quantified error analysis, and a retrieval experiment comparing stemming against full analysis.

TASK 10 – Implement and Test the Pipeline 3 Marks

Guiding Enquiry

Can a working implementation reproduce the behaviour you derived by hand?

Investigate and interpret:

- Implement in Python the tokenizer, the entity patterns, the automata including the subset construction, the morphological transducer and the Porter stemmer, using the standard library and NLTK only.
- Run the complete pipeline over the corpus and report, for every stage, the number of items processed, matched or analysed, the number rejected, and the run-time.
- Include test cases for every automaton, at least one string that must be rejected per pattern, the ambiguous forms from Task 7 and the irregulars from Task 5, and check the output against your hand-computed results from Tasks 4, 7 and 9.
- Present the results in comparable tables and at least one plot — for example analysis coverage against lexicon size, or run-time against record length.

Evidence Expected

Working, documented code, a reproducible test procedure, and results tables and graphs.

TASK 11 – Compare, Recommend and Defend the Normalisation Design 2 Marks

Guiding Enquiry

Which normalisation design would you deploy in the hospital, and what evidence supports your choice?

Investigate and interpret:

- Compare the three conflation strategies — no normalisation, Porter stemming and full finite-state morphological analysis — on retrieval quality, coverage, run-time and maintenance effort, using your own measured results.
- Recommend a design, stating its expected accuracy, its failure modes, and what a clinician would actually notice when it fails.
- Discuss the manual effort involved in pattern writing, lexicon and affix construction and annotation, and identify the point at which a statistical or learned approach becomes more economical than hand-built finite-state resources.

Evidence Expected

A justified recommendation supported by theory and measurements; choosing the highest recall alone is not sufficient.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement

The system must now ingest two new sources. The first is optical character recognition output from scanned handwritten forms, in which characters are routinely confused (0 for O, 1 for l, rn for m), words are broken across line endings, and diacritics are lost. The second is notes typed by staff in Romanised regional language mixed with English, for example “patient ku fever irundhuchu, paracetamol 500mg kuduthen”. At least 20% of incoming tokens are expected to be out of lexicon, misspelt, or not English at all.

Re-evaluate your original design under the changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Guiding Enquiry

What exactly has changed, and which of your original assumptions no longer hold?

- Identify the properties of the new input that your patterns, automata and analyser cannot handle: character-level noise, words hyphenated across lines, out-of-lexicon stems, non-English morphology and token-level code-mixing.
- Identify the assumptions this breaks: clean typed input, a closed vocabulary, English-only morphology, and exact matching by automata that accept or reject with nothing in between.
- Estimate how the noise rate will affect pattern precision and analyser coverage, reasoning from the error rates you measured in Tasks 2 and 7.

Evidence Expected

A clear analysis of how the new input exposes gaps in the original design.

B. Evaluation of the Original Design – 2 Marks

Guiding Enquiry

How would your recommended pipeline actually behave on the new input?

- Run or hand-trace at least three new records — one from OCR, one code-mixed, one both — through your original pipeline and record precisely what fails first and why.
- Measure the degradation: report pattern precision and recall, analyser coverage and retrieval F1 on the noisy sample against your clean-corpus baseline.
- Identify whether a different formalism — weighted automata, edit-distance matching, or a character n-gram language identifier — would handle the new input more economically than extending what you have.

Evidence Expected

A technically supported evaluation of the original design with traces or measurements.

C. Adaptation or Redesign – 2 Marks

Guiding Enquiry

Should the finite-state layer be extended, replaced or supplemented?

- Choose retain, modify or redesign — for example placing a weighted edit-distance transducer in front of the analyser, extending the lexicon with regional stems, adding

token-level language identification, or falling back to stemming for out-of-lexicon tokens.

- Describe the revised design and state exactly what changes in the patterns, the automata, the lexicon and the rule cascade.
- Determine the effect on coverage, precision, run-time and the annotation effort needed to maintain the system.

Evidence Expected

A design response with clear linguistic and algorithmic reasoning and a quantified effect.

D. Evidence-Based Defence – 2 Marks

Guiding Enquiry

Why is your final decision the right one, given the evidence you generated?

- Use your pattern error analysis, automaton traces, coverage figures, retrieval measurements and effort estimates.
- Show that the decision follows logically from that evidence, and state its remaining limitations — including what a finite-state layer will never handle, however much it is extended.

Evidence Expected

A well-supported final decision integrating theory, experiments and the changed requirements.

MARK DISTRIBUTION

Task	Marks
Task 1 – Frame the Problem: Models, Algorithms and Levels of Analysis	3
Task 2 – Basic Regular Expression Patterns	4
Task 3 – Regular Expressions for Tokenisation and Normalisation	4
Task 4 – Finite State Automata: Design, Non-determinism and Equivalence	5
Task 5 – Inflectional Morphology	3
Task 6 – Derivational Morphology	4
Task 7 – Finite-State Morphological Parsing	5
Task 8 – Orthographic Rules and Two-Level Morphology	4
Task 9 – The Porter Stemmer	5
Task 10 – Implement and Test the Pipeline	3
Task 11 – Compare, Recommend and Defend the Normalisation Design	2
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**

ABSTRACT

Hospitals store clinical information in discharge summaries, prescriptions, operation notes and nursing observations. Exact keyword searches may miss related forms such as fracture, fractures and fractured. This assignment proposes a text normalization and search layer using regular expressions, finite-state automata, finite-state transducers and the Porter stemmer. It also considers noisy OCR and English–Tamil code-mixed notes. The examples demonstrate the design; corpus-based results must be measured on the prepared dataset.

PROBLEM STATEMENT

The hospital requires a preprocessing layer that recognizes structured clinical entities, tokenizes and normalizes free text, relates inflected and derived forms, and improves retrieval. The system must be explainable, preserve the original text and adapt to noisy input.

DATASET AND EXPERIMENTAL NOTE

Prepare at least 500 de-identified synthetic or approved records containing at least 25,000

tokens and four types: discharge summaries, prescriptions, operation notes and nursing observations. Prepare 40 clinician queries with relevance judgements and hand-annotate 200 word forms. Do not report illustrative values as measured results. Fill every [MEASURE] field after running the experiment.

Item	Required quantity
Records	At least 500
Tokens	At least 25,000
Record types	At least 4
Queries	At least 40
Annotated word forms	At least 200

TASK 1 – FRAME THE PROBLEM: MODELS, ALGORITHMS AND LEVELS OF ANALYSIS

1.1 Levels of linguistic analysis

Level	Description and clinical example	Need
Phonology	Sound patterns; pronunciation variants of a medicine name	Not central for typed records; useful for speech input
Morphology	Word structure; fractures = fracture + plural	Essential
Syntax	Word order; “Patient was given medicine”	Useful for advanced relation extraction
Semantics	Meaning; renal failure differs from heart failure	Important for safe search
Pragmatics	Context and intention; “rule out pneumonia” is not confirmed pneumonia	Important for clinical interpretation
Discourse	Links across sentences and sections	Useful for advanced record understanding

The core project requires morphology and structured-token recognition. Syntax, semantics, pragmatics and discourse are not fully implemented in the finite-state layer, but clinical deployment must account for negation and uncertainty. Phonology can be deferred unless speech input is added.

1.2 Model families

Model family	Best hospital task	Reason
State machines	Validate patient IDs, dates and fixed entity formats	They recognise permitted symbol sequences exactly.
Formal rule systems	Apply suffix rules and abbreviation expansion	They are interpretable and deterministic.
Probabilistic models	Rank OCR corrections and ambiguous interpretations	They assign likelihoods to uncertain alternatives.

A probabilistic model is unnecessary for a fixed ID format. A DFA is unsuitable for ranking uncertain OCR alternatives. A rule list alone is insufficient to rank competing interpretations.

1.3 Ambiguity in clinical text

- Lexical ambiguity: “lead” may refer to a metal, a cardiac lead or a verb.
- Morphological ambiguity: “wound” may be a noun or the past tense of “wind”.
- Segmentation ambiguity: “500mg” may be treated as one token or as 500 plus mg.
- Abbreviation ambiguity: “MS” may mean multiple sclerosis, morphine sulphate or medical student.

1.4 Search and resource use

Automata explore states while consuming input symbols. DFA recognition takes O(n) time for a string of length n. NFA simulation may maintain a set of active states. Backtracking regex engines can take much longer on poorly designed patterns. Memory is used for states, transitions and input. Annotated data supports evaluation, while expert effort is required for lexicon construction, pattern writing and clinical review.

1.5 Pipeline

Clinical records → Regex tokenisation → Entity matching / DFA validation → Case and abbreviation normalisation → Morphological FST → Porter fallback → Search index → Ranked records

Input: de-identified free-text clinical records. Output: recognised entities, normalised tokens, lemma/features and searchable record references. The original text and offsets are retained.

TASK 2 – BASIC REGULAR EXPRESSION PATTERNS

2.1 Basic operators

Operator	Meaning	Example
Concatenation	Sequence	[0-9][0-9]
Disjunction |	Either alternative	mg|mcg
Kleene star *	Zero or more	[0-9]*
Plus +	One or more	[0-9]+

Optional ?	Zero or one	-?
Character class	One character from a set	[A-Z]
Range	Continuous range	[0-9]
Negated class	Any character except listed	[^\n]+
Anchors	Start and end	^ID[0-9]+$
Counters	Fixed repetition	[0-9]{4}

2.2 Twelve clinical entity patterns

Patient identifier: \bPT-[0-9]{6}\b

Matches PT-123456; rejects five or seven digits.

Admission/discharge date: \b(?:[0-3][0-9]/[01][0-9]/[0-9]{4}|[0-9]{4}-[01][0-9]-[0-3][0-9])\b

Recognises DD/MM/YYYY or YYYY-MM-DD shapes; calendar validity needs a date library.

Drug with strength: \b[A-Za-z][A-Za-z-]*\s+[0-9]+(?:\.[0-9]+)?\s?(?:mg|mcg|g|ml|mL)\b

Matches Paracetamol 500 mg; rejects strength without a drug name.

Dosage frequency: \b(?:OD|BD|TDS|QID|once daily|twice daily|every [0-9]+ hours)\b

Matches listed frequencies; does not infer other schedules.

Lab value and unit: \b[A-Za-z][A-Za-z]{0,25}:?\s*[0-9]+(?:\.[0-9]+)?\s?(?:mg/dL|g/dL|mmol/L|U/L)\b

Matches Glucose: 110 mg/dL; rejects unlisted units.

Blood pressure: \b[0-9]{2,3}/[0-9]{2,3}\s?mmHg\b

Matches 120/80 mmHg; clinical plausibility is separate.

Temperature: \b(?:3[0-9]|4[0-5])(?:\.[0-9])?\s?°?[CF]\b

Matches 37.0 C; range checking is separate.

ICD code: \b[A-Z][0-9]{2}(?:\.[A-Z0-9]{1,4})?\b

Matches E11.9; validate against the relevant ICD release.

Ward and bed: \bWard\s+[A-Z0-9-]+,?\s+Bed\s+[A-Z0-9-]+\b

Matches Ward A, Bed 12; rejects missing bed.

Contact number: \b(?:\+91[-]?)?[6-9][0-9]{9}\b

Matches Indian mobile number format.

Age: \b[0-9]{1,3}\s?(?:years?|yrs?|months?|days?)\b

Matches 65 years or 2 months; range validation is separate.

Symptom duration: \b(?:[0-9]+|one|two|three|four|five)\s?(?:hours?|days?|weeks?|months?)\b

Matches 3 days or two weeks; context is not guaranteed.

These patterns recognise formats, not complete clinical truth. Date validity, drug dictionary membership, ICD validity and plausible vital-sign ranges need separate checks.

2.3 Greedy and non-greedy matching

```
text = "Give 500 mg paracetamol, then 250 mg after 6 hours."\nre.search(r".*mg", text)   # greedy: extends to
```

```
last mg\nre.search(r".*?mg", text)  # non-greedy: stops at first mg
```

Greedy matching consumes as much as possible while allowing the expression to match. Non-greedy matching consumes as little as possible. Use a specific dosage pattern with finditer to extract each dose reliably.

2.4 Precedence and grouping

```
Wrong: ^Ward|Bed[0-9]+$\nRight: ^(?:Ward|Bed)[0-9]+$
```

In the wrong expression, alternation applies to the two large alternatives. Grouping constrains the alternatives to the intended shared structure.

2.5 Error evaluation

Manually label true entity spans. TP means correct detections, FP means incorrect detections and FN means missed entities. Precision = TP/(TP+FP); recall = TP/(TP+FN). Report per-pattern results. Causes include OCR errors, unseen abbreviations, spacing variation and invalid dates.

TASK 3 – REGULAR EXPRESSIONS FOR TOKENISATION AND NORMALISATION

3.1 Token definition

A token is a clinically meaningful word, abbreviation, number, decimal, hyphenated term or measurement unit. Dotted abbreviations and decimals should remain intact. A number and unit may be separate tokens while retaining the original measurement span.

```
TOKEN_RE = re.compile(
 r"[A-Za-z]+(?:[.-][A-Za-z]+)*\.?"
 r"|\d+(?:\.\d+)?"
 r"|[^\w\s]", re.IGNORECASE
)
```

Use ordered alternatives and maximal-munch behaviour. A measurement-aware pass can identify and preserve number–unit pairs.

3.2 Maximal-munch examples

Input	Naive whitespace split	Token-aware result
BP: 120/80 mmHg	BP:, 120/80, mmHg	BP | : | 120 | / | 80 | mmHg; BP span retained
Take 0.5 mg	Take, 0.5, mg	Take | 0.5 | mg; decimal remains one token
Dr. Ravi prescribed 500mg	Dr., Ravi, prescribed, 500mg	Dr. | Ravi | prescribed | 500 | mg

3.3 Abbreviation expansion

Abbreviation	Expansion
BP	blood pressure
HR	heart rate
RR	respiratory rate
SOB	shortness of breath
c/o	complains of
h/o	history of
DM	diabetes mellitus
HTN	hypertension
CVA	cerebrovascular accident
MI	myocardial infarction
CKD	chronic kidney disease
AKI	acute kidney injury
UTI	urinary tract infection
COPD	chronic obstructive pulmonary disease
CAD	coronary artery disease
OD	once daily
BD	twice daily
TDS	three times daily
QID	four times daily
SOS	if required
PRN	as needed
NPO	nothing by mouth
IV	intravenous
IM	intramuscular
PO	by mouth
ECG	electrocardiogram
CBC	complete blood count
RBS	random blood sugar
FBS	fasting blood sugar
SpO2	oxygen saturation

3.4 Worked normalisation

Before: “PT-123456 admitted on 05/11/2026. c/o fever for 2 days. BP 120/80 mmHg. Tab.

Paracetamol 500mg BD.”

After: “pt-123456 admitted on 2026-11-05. complains of fever for 2 days. blood pressure 120/80 mmHg. tablet paracetamol 500 mg twice daily.”

The date conversion assumes DD/MM/YYYY. If the source convention is unknown, the date must not be converted automatically.

3.5 Irreversible decisions and limits

- Case folding loses capitalisation. Preserve the original text for display.
- Abbreviation expansion loses the writer’s exact shorthand. Keep original and expanded forms.
- Unit conversion may introduce rounding. Retain original value and unit.

Regular expressions cannot recognise arbitrarily nested balanced parentheses because that language requires unbounded counting of nesting depth. A stack-based parser is required.

TASK 4 – FINITE STATE AUTOMATA: DESIGN, NON-DETERMINISM AND EQUIVALENCE

4.1 Formal definition

A finite automaton is the five-tuple $M = (Q, \Sigma, \delta, q_0, F)$, where Q is a finite set of states, Σ is the input alphabet, δ is the transition function, q_0 is the initial state and F is the set of accepting states.

4.2 DFA validator designs

Validator	State sequence / transition summary	Accepting condition
Patient ID	q0 -P→ q1 -T→ q2 --> q3; six digit transitions q3→q4→q5→q6→q7→q8→q9	Exactly six digits consumed
Age	q0 digit→q1; q1 loops on digits; space→q2; then y→e→a→r→s	Valid age unit completed
Blood pressure	Consume 2–3 systolic digits; slash; consume 2–3 diastolic digits; optional space; mmHg	Unit completed
ICD code	Uppercase letter; two digits; optional dot and 1–4 alphanumeric suffix	Base code or suffix completed

A sink state receives all invalid symbols. Example patient-ID transitions:

Current state	Input	Next state
q0	P	q1
q1	T	q2
q2	-	q3
q3	digit	q4
q4	digit	q5
q5	digit	q6
q6	digit	q7
q7	digit	q8
q8	digit	q9 (final)
Any	invalid symbol	sink

4.3 NFA with epsilon transition

Use an identifier format with an optional site letter: PT-[A-Z]-123456 or PT-123456. After reading PT-, the NFA branches: one path takes an epsilon transition to the digit sequence; the other consumes a letter and hyphen before the digits.

q0 -P→ q1 -T→ q2 -hyphen→ q3\nq3 --ε--> q4 (digit sequence)\nq3 --A-Z--> q5 --hyphen--> q4\nq4 --digit--> q6 --digit--> q7 --digit--> q8 --digit--> q9 --digit--> q10 --digit--> q11 (final)

Accepted: PT-A-123456 follows the letter branch and then six digit transitions. Rejected: PT-A-12 stops before six digits. The active-state set at each input position is the epsilon-closure of all reachable states.

4.4 Subset construction and minimisation

Each DFA state represents a set of NFA states. Begin with epsilon-closure({q0}). For each symbol, compute the union of reachable NFA states and take its epsilon-closure. A subset is accepting if it contains an NFA final state. Minimise by partition refinement: separate final and non-final states, then split blocks whose transitions reach different blocks. Record the actual NFA, DFA and minimised state counts from your implementation; these depend on the precise transition graph.

4.5 Thompson construction and equivalence

Thompson's construction creates an epsilon-NFA fragment for each symbol, then combines fragments for concatenation, alternation and Kleene star. For PT-[0-9]{6}, create literal P, literal T, hyphen and six digit transitions. State elimination derives a regular expression from an automaton: add new start/final states and eliminate intermediate states, updating edge labels with $R_{ij} \cup R_{ik}(R_{kk})^*R_{kj}$. Both constructions preserve the accepted language.

4.6 Pumping lemma

For every regular language there is a pumping length p such that every string s in the language

with length at least p can be written s=xyz, with $|xy|\leq p$, $|y|>0$ and xy^iz in the language for all $i\geq 0$.

Consider $L = \{ '('^n ')'^n : n\geq 0 \}$. Assume it is regular. Choose $s='('^p')'^p$. Since $|xy|\leq p$, y contains only opening parentheses. Pump y to zero: the resulting string has fewer opening than closing parentheses and is not in L. This contradiction proves that balanced parentheses are not regular.

TASK 5 – INFLECTIONAL MORPHOLOGY

5.1 Definitions

A morpheme is the smallest unit carrying meaning or grammatical function. A stem is the form to which an affix attaches. An affix is a bound morpheme attached to a stem. Inflection changes grammatical features without normally changing word class. A paradigm is the set of forms of a lexeme.

Inflection is generally productive and category-preserving: fracture → fractures. Derivation may change meaning or category: operate → operation. Derivational affixes are normally closer to the root than inflectional endings.

5.2 Clinical paradigms

Lexeme	Noun forms	Verb forms / other forms
fracture	fracture / fractures	fracture / fractured / fracturing
treat	treatment / treatments	treat / treats / treated / treating; treatable
infect	infection / infections	infect / infects / infected / infecting; infectious
bleed	bleed / bleeds	bleed / bleeds / bled / bleeding
swell	swelling / swellings	swell / swells / swelled / swelling

Subject agreement appears in patient improves / patients improve. Adjective comparison may occur in mild, milder and mildest.

5.3 English spelling rules

- E-deletion: create + ing → creating. Counter-example: dye + ing → dyeing; deleting e would produce “dying”, a different word.
- Consonant doubling: admit + ed → admitted. Counter-example: visit + ed → visited, not visitted.
- Y-to-i: carry + ed → carried. Counter-example: play + ed → played because y follows a vowel.

- Sibilant -es: wash + s → washes. Counter-example: quiz → quizzes also requires consonant doubling.

5.4 Irregular forms and measurements

List irregular forms separately: bleed/bled, give/gave/given, take/took/taken, be/was/been, go/went/gone and have/has/had. Type-to-lemma ratio = distinct surface word types / distinct lemmas. Inflection-related query failure rate = queries recovered solely by inflectional expansion / queries that initially failed. Calculate both from the corpus.

TASK 6 – DERIVATIONAL MORPHOLOGY

6.1 Derivational affixes

Affix	Example	Meaning / category effect
-itis	arthritis	Inflammation; noun
-ectomy	nephrectomy	Surgical removal; noun
-ology	cardiology	Field of study; noun
-osis	fibrosis	Condition; noun
-aemia	anaemia	Blood condition; noun
-ation	amputation	Action/process; noun
-able	treatable	Adjective
-ive	sedative	Adjective or noun
nephro-	nephrology	Kidney-related combining form
hyper-	hypertension	High / excessive
hypo-	hypoglycaemia	Low / deficient
re-	reoperation	Again
un-	unhealed	Negation / reversal
-al	renal	Adjective
-ic	cardiac	Adjective
-ist	specialist	Person / profession noun

6.2 Derivational families

Root	Family
nephr	nephr + o + logy → nephrology; nephro + ectomy → nephrectomy
cardi	cardi + ac → cardiac; cardi + o + logy → cardiology
infect	infect + ion → infection; infect + ious → infectious
operate	operate → operation; re + operation → reoperation
treat	treat + ment → treatment; treat + able → treatable
haem	haem + orrhage → haemorrhage; haem + at + ology → haematology

Affix order is constrained. Plural -s normally attaches outside a noun-forming suffix: infection + s → infections. *-s + -ation is not a valid derivation of a noun from a verb. *nephro + -itis + -ology is not a standard formation of nephrology.

6.3 Non-compositionality and search

A derived word's meaning is not always predictable from its parts. "Heartburn" does not mean literal burning of the heart. Aggressive stemming may create misleading matches. Renal and nephrectomy are related but not interchangeable; infection and infectious have different grammatical roles.

6.4 Productivity and conflation

Count distinct derived types and total tokens for every affix. An affix occurring with many different roots is more productive; one found in a few fixed words may be fossilised. Conflation helps when a search for fracture should find fractures, or infection should find infections. It may mislead when renal retrieves every nephrectomy mention, or treatment is treated as identical to treatable. Link derivational relatives as related concepts rather than identical lemmas.

TASK 7 – FINITE-STATE MORPHOLOGICAL PARSING

7.1 Finite-state transducer

A finite-state transducer (FST) is a finite-state machine with an input tape and an output tape. In analysis, it reads a surface form and emits a lexical representation. In generation, it reads lexical input and produces a surface form. The same relation can be traversed in opposite

directions.

Example: fractures maps to fracture+N+PL. The stem corresponds to fracture:fracture and the ending s emits the plural feature.

7.2 Lexicon continuation classes

Class	Entries	Continuation
Noun	fracture, infection, patient, kidney	Plural -s and licensed derivational suffixes
Verb	treat, admit, fracture, bleed	3SG -s, past -ed, progressive -ing; irregular table
Adjective	renal, cardiac, severe	Comparative -er and superlative -est where applicable
Irregular	bleed:bled, go:went	Explicit surface/lexical mapping

7.3 Five analyses

Surface	Lexical analysis	Features
fractures	fracture + N + PL	Noun, plural
fractured	fracture + V + PAST	Verb, past
treating	treat + V + PROG	Verb, progressive
infections	infection + N + PL	Noun, plural
bled	bleed + V + PAST	Irregular past

For a full hand trace, list the state sequence, symbols consumed on the surface tape and symbols emitted on the lexical tape. The FST follows the stem path, enters the relevant continuation class and emits grammatical tags.

7.4 Generation and ambiguity

Generation reverses the relation: fracture+N+PL → fractures; treat+V+PAST → treated; bleed+V+PAST → bled. It is the same transducer relation, not a separate grammar.

“Wound” may be wound+N+SG or wind+V+PAST. “Treated” may be treat+V+PAST or a participial adjective. The FST returns all licensed analyses; a contextual component ranks them. Alternatives should not be silently discarded.

TASK 8 – ORTHOGRAPHIC RULES AND TWO-LEVEL MORPHOLOGY

8.1 Need for orthographic rules

Plain concatenation produces incorrect forms: create + ing → createing instead of creating; carry + ed → carryed instead of carried; admit + ed → admited instead of admitted. Orthographic rules map lexical sequences to valid surface spellings.

8.2 Rewrite rules

Rule (informal notation)	Example
e → ∅ / _ + vowel suffix	create+ing → creating
y → i / consonant _ + ed	carry+ed → carried
y → i / consonant _ + es	carry+es → carries
C → CC / stressed CVC _ + vowel suffix	admit+ed → admitted
Insert e before -s after sibilant	wash+s → washes

The context must include morpheme boundaries. Consonant doubling also depends on stress and syllable structure, so spelling-only rules are not sufficient for every English word.

8.3 Cascade and two-level approach

Lexicon FST → e-deletion → y-to-i → consonant doubling → sibilant rule → surface form

Rule order matters because an earlier rule may change the context seen by a later rule. A sequential cascade is easy to test and debug. Koskenniemi two-level morphology enforces lexical–surface correspondence constraints in parallel, reducing some order dependencies but increasing implementation complexity.

8.4 Testing

Test at least 20 forms, including create/creating, admit/admitted, carry/carried, carry/carries, wash/washes, study/studied, play/played, stop/stopped, treat/treated and treat/treating. Record expected form, generated form, pass/fail and failure cause.

TASK 9 – THE PORTER STEMMER

9.1 Algorithm definitions

The Porter stemmer is a rule-based suffix-stripping algorithm. Vowels and consonants are defined over a word's letter sequence; y may behave as either depending on context. The measure m counts vowel–consonant sequences in the stem. The conditions *v*, *d and *o mean respectively that the stem contains a vowel, ends in a double consonant, or ends in consonant-vowel-consonant with final consonant not w, x or y.

9.2 Measure and hand traces

Calculate m for six clinical words using the exact Porter definitions and show the VC grouping. Then trace the five ordered stages: (1) plural and -ed/-ing handling; (2) longer derivational

suffix replacements; (3) further suffix replacements; (4) removal of selected endings under measure conditions; (5) final -e removal and final double-l adjustment. The intermediate result after each stage must be recorded.

Clinical word	Stem output from NLTK	Stage trace
fractures	[MEASURE]	[FILL EACH STAGE]
fractured	[MEASURE]	[FILL EACH STAGE]
infections	[MEASURE]	[FILL EACH STAGE]
treatments	[MEASURE]	[FILL EACH STAGE]
admissions	[MEASURE]	[FILL EACH STAGE]
bleeding	[MEASURE]	[FILL EACH STAGE]

9.3 Stemming versus lemmatisation

Stemming strips suffixes and may produce a non-word. Lemmatisation uses lexical and morphological knowledge to return a dictionary lemma and features. Compare both methods on 200 annotated forms and count agreement, divergence, over-stemming and under-stemming.

9.4 Retrieval comparison

Method	Precision	Recall	F1	Runtime
No stemming	[MEASURE]	[MEASURE]	[MEASURE]	[MEASURE]
Porter stemming	[MEASURE]	[MEASURE]	[MEASURE]	[MEASURE]
Full morphology	[MEASURE]	[MEASURE]	[MEASURE]	[MEASURE]

Over-stemming merges unrelated forms and creates irrelevant matches. Under-stemming leaves related forms separate and causes missed records. Measure both on the annotated forms and 40 query judgements.

TASK 10 – IMPLEMENT AND TEST THE PIPELINE

10.1 Python starter implementation

The following code demonstrates tokenisation, entity matching, abbreviation normalisation and Porter stemming. Extend it with the DFA/NFA and FST implementations required in Tasks 4 and 7.

```
import re
from nltk.stem import PorterStemmer

PATTERNS = {
 "patient_id": re.compile(r"\bPT-[0-9]{6}\b", re.I),
 "date": re.compile(r"\b(?:[0-3]\d/[01]\d/\d{4}|\d{4}-[01]\d-[0-3]\d)\b"),
 "drug_strength": re.compile(r"\b[A-Za-z][A-Za-z-]*\s+\d+(?:\.\d+)?\s?(?:mg|mcg|g|ml|mL)\b", re.I),
 "frequency": re.compile(r"\b(?:OD|BD|TDS|QID|PRN)\b", re.I),
```

```
 "blood_pressure": re.compile(r"\b\d{2,3}/\d{2,3}\s?mmHg\b", re.I),
 "temperature": re.compile(r"\b(?:3\d|4\d)(?:\.\d)?\s?°?[CF]\b", re.I),
 "icd": re.compile(r"\b[A-Z]\d{2}(?:\.[A-Z0-9]{1,4})?\b", re.I),
 "contact": re.compile(r"\b(?:\+91[- ]?)?[6-9]\d{9}\b"),
 "age": re.compile(r"\b\d{1,3}\s?(?:years?|yrs?|months?|days?)\b", re.I),
 "duration": re.compile(r"\b\d+\s?(?:hours?|days?|weeks?|months?)\b", re.I)
}
TOKEN_RE = re.compile(r"[A-Za-z]+(?:[.-][A-Za-z]+)*\.?|\d+(?:\.\d+)?|[^\w\s]")
ABBR = {"BP":"blood pressure", "HR":"heart rate", "SOB":"shortness of breath",
        "DM":"diabetes mellitus", "HTN":"hypertension", "OD":"once daily",
        "BD":"twice daily", "TDS":"three times daily", "QID":"four times daily"}
stemmer = PorterStemmer()

def analyse_record(text):
    entities = {name: pat.findall(text) for name, pat in PATTERNS.items()}
    tokens = TOKEN_RE.findall(text)
    normalised = [ABBR.get(tok.upper(), tok.casefold()) for tok in tokens]
    stems = [stemmer.stem(tok) for tok in normalised
             if re.fullmatch(r"[a-z]+", tok)]
    return entities, tokens, normalised, stems

sample = "PT-123456 admitted 05/11/2026. BP 120/80 mmHg. Paracetamol 500 mg BD."
print(analyse_record(sample))
```

10.2 Test reporting

Test every automaton with accepted and rejected strings, each regex with at least one negative example, all irregular forms and the ambiguous FST forms. Compare outputs against hand calculations.

Stage	Processed	Matched / analysed	Rejected	Runtime
Tokenizer	[MEASURE]	[MEASURE]	[MEASURE]	[MEASURE]
Entity patterns	[MEASURE]	[MEASURE]	[MEASURE]	[MEASURE]
Automata	[MEASURE]	[MEASURE]	[MEASURE]	[MEASURE]
Morphological analyser	[MEASURE]	[MEASURE]	[MEASURE]	[MEASURE]
Porter stemmer	[MEASURE]	[MEASURE]	[MEASURE]	[MEASURE]

Include at least one plot, such as runtime against record length or analysis coverage against lexicon size. Use the same corpus and hardware for fair comparisons.

TASK 11 – COMPARE, RECOMMEND AND DEFEND THE DESIGN

Design	Advantages	Limitations
No normalisation	Preserves original wording; simple	Misses inflectional variants
Porter stemmer	Fast; no lexicon needed	May over-stem and produce non-words
Finite-state morphology	Interpretable; handles features and listed irregulars	Requires expert-built lexicon and rules

Recommendation: use a hybrid design. Apply regex and DFA validation to structured entities, then finite-state morphology for known clinical vocabulary. Preserve original text and all analyses. Use Porter stemming only as a low-confidence fallback for unknown English words, never as the sole clinical equivalence decision. Link derivational relatives rather than automatically treating them as identical.

Report expected accuracy from the actual experiment. Failure modes include unseen abbreviations, OCR corruption, ambiguous words, code-mixed morphology and missing context. Manual work includes pattern writing, a 300-stem lexicon, at least 20 inflectional and 15 derivational affixes, 200 annotated forms and 40 judged queries. A learned method may become more economical when vocabulary variation makes manual updates more expensive than collecting representative labelled examples.

TASK 12 – DAY-4 DESIGN CHALLENGE

A. Impact analysis

The new input contains OCR confusions (0/O, 1/l, rn/m), words broken across line endings, lost diacritics, out-of-lexicon words and Romanised regional language mixed with English. The assumptions of clean typed input, closed vocabulary, English-only morphology and exact accept/reject matching are no longer valid.

Exact patterns may miss corrupted identifiers, dates and drug strengths. The English analyser will reject regional-language tokens. For example, “kuduthen” will not be analysed by an English suffix lexicon. At least 20% noisy or out-of-lexicon tokens will reduce exact-match coverage.

B. Evaluate the original design

New input	Original pipeline behaviour	First likely failure
OCR: PT-12O456 fractured	ID pattern rejects letter O in digit position; fractured may analyse	Exact entity matching
Code-mixed: patient ku fever	English words may pass; ku	English-only analyser

irundhuchu	and irundhuchu unknown	
Mixed: paracetamoI 5OOmg kuduthen	Drug pattern may reject OCR substitutions; regional verb unknown	Entity recognition and lexicon lookup

Run these records through the original implementation. Measure pattern precision/recall, analyser coverage and retrieval F1 against the clean baseline.

Metric	Clean baseline	Noisy sample	Change
Pattern precision	[MEASURE]	[MEASURE]	[CALCULATE]
Pattern recall	[MEASURE]	[MEASURE]	[CALCULATE]
Analyser coverage	[MEASURE]	[MEASURE]	[CALCULATE]
Retrieval F1	[MEASURE]	[MEASURE]	[CALCULATE]

Weighted automata or edit-distance matching can rank likely OCR corrections. Character n-gram language identification can identify likely English versus Romanised regional tokens. These methods handle uncertainty more naturally than adding every possible OCR error as an exact pattern.

C. Adaptation or redesign

Decision: MODIFY the original system. Retain the finite-state core and add a noise-tolerant front end.

1. OCR correction: use weighted edit-distance candidates for O/0, l/1 and rn/m. Correct automatically only above a conservative confidence threshold; otherwise flag for review.
2. Line-break repair: join hyphenated words across lines only when layout and dictionary evidence support the join. Preserve original text.
3. Language identification: use character n-grams to label tokens as English, Romanised Tamil, mixed, numeric or unknown.
4. Lexicon extension: add reviewed regional-language stems and common code-mixed forms with language tags.
5. Entity patterns: keep strict validators, but run candidate correction before validation and retain raw and corrected spans.
6. Fallback: use Porter stemming only for likely English unknown words. Do not apply English stemming to Romanised Tamil.

Coverage should improve, but precision may fall if correction is aggressive. Confidence thresholds, audit trails and clinician review are required. Quantify the effects on the noisy test set.

D. Evidence-based defence

The modified design retains exact finite-state recognition for reliable input and introduces weighted correction and language identification for uncertain input. Defend the decision using measured clean/noisy precision, recall, F1, coverage, runtime and annotation effort. Remaining limitations include unseen regional languages, severe handwriting errors, ambiguous abbreviations, context-dependent meaning, negation and nested structures. A finite-state layer cannot fully understand clinical intent or replace clinician judgement.

CONCLUSION

This assignment presented a finite-state and morphology-aware method for clinical text normalisation. Regular expressions identify structured patterns, automata validate formats, morphological analysis relates surface forms to lemmas and features, and Porter stemming provides a lightweight fallback. OCR noise and code-mixed language require a confidence-aware extension. The recommended design is hybrid and conservative: preserve original text, expose uncertainty and avoid treating related clinical terms as identical.

REFERENCES

- Jurafsky, D. and Martin, J. H., Speech and Language Processing, 3rd ed. draft.
- Porter, M. F. (1980). An algorithm for suffix stripping. Program, 14(3), 130–137.
- Hopcroft, J. E., Motwani, R. and Ullman, J. D., Introduction to Automata Theory, Languages, and Computation.
- Koskenniemi, K. (1983). Two-level Morphology: A General Computational Model for Word-Form Recognition and Production.
- Bird, S., Klein, E. and Loper, E. (2009). Natural Language Processing with Python.